Εφαρμογή Διαχείρισης Εκδηλώσεων και Ηλεκτρονικών Κρατήσεων

Μάθημα: Τεχνολογίες Εφαρμογών Διαδικτύου
Ομάδα: Ραφαέλα Σιώρου - sdi2300186, Παντελής Κοντάρης - sdi2300079
Τμήμα: Πληροφορικής και Τηλεπικοινωνιών
Ημερομηνία: 27/09/2026

Πίνακας Περιεχομένων

1. Εισαγωγή
2. Αρχιτεκτονική Εφαρμογής
3. Σχεδίαση Βάσης Δεδομένων
4. Πιστοποίηση και Εξουσιοδότηση
5. Διαχείριση Χρηστών
6. Διαχείριση Εκδηλώσεων
7. Κρατήσεις Εισιτηρίων
8. Σύστημα Μηνυμάτων
9. Αναζήτηση, Χάρτες και Φωτογραφίες
10. Εξαγωγή Δεδομένων σε XML/JSON
11. Σύστημα Συστάσεων
12. Εγκατάσταση και Εκτέλεση
13. Έλεγχος και Δοκιμές
14. Συμπεράσματα και Μελλοντικές Βελτιώσεις

---

1. Εισαγωγή

Σκοπός της εργασίας ήταν να αναπτύξουμε μία διαδικτυακή εφαρμογή για διαχείριση εκδηλώσεων και ηλεκτρονικές κρατήσεις εισιτηρίων. Η εφαρμογή υποστηρίζει διαφορετικούς ρόλους χρηστών: επισκέπτες, εγγεγραμμένους χρήστες/συμμετέχοντες, διοργανωτές και διαχειριστές.

Οι επισκέπτες μπορούν να βλέπουν και να αναζητούν δημοσιευμένες εκδηλώσεις, αλλά δεν μπορούν να κάνουν κράτηση χωρίς σύνδεση. Οι εγγεγραμμένοι χρήστες μπορούν

να κάνουν κρατήσεις και να επικοινωνούν με διοργανωτές. Οι διοργανωτές μπορούν να δημιουργούν και να διαχειρίζονται εκδηλώσεις. Ο διαχειριστής εγκρίνει χρήστες και έχει πρόσβαση σε λειτουργίες διαχείρισης και εξαγωγής δεδομένων.

Η εφαρμογή υλοποιήθηκε με αρχιτεκτονική web browser / web server. Το frontend αναπτύχθηκε με Angular και το backend με NestJS. Για τη βάση δεδομένων χρησιμοποιήθηκε PostgreSQL, με Prisma ORM για την αντιστοίχιση του σχεσιακού μοντέλου σε αντικειμενοστρεφές μοντέλο μέσα στον κώδικα.

Στα επόμενα κεφάλαια περιγράφουμε τις βασικές λειτουργίες, τις σχεδιαστικές επιλογές, τον τρόπο εγκατάστασης και εκτέλεσης, καθώς και τα σημεία που ελέγχθηκαν κατά την ανάπτυξη.

2. Αρχιτεκτονική Εφαρμογής

Η εφαρμογή χωρίζεται σε δύο βασικά projects:

- `frontend` : Angular εφαρμογή για το UI.
- `backend` : NestJS REST API για τη λογική της εφαρμογής και την επικοινωνία με τη βάση.

Το frontend καλεί το backend μέσω HTTP requests. Τα protected requests στέλνουν JWT token στο `Authorization` header. Το backend ελέγχει το token και αποφασίζει αν ο χρήστης έχει δικαίωμα να εκτελέσει την αντίστοιχη ενέργεια.

Στο backend ακολουθήθηκε η δομή του NestJS με modules, controllers και services. Τα βασικά modules είναι:

- `AuthModule`
- `UsersModule`
- `EventsModule`
- `BookingsModule`
- `MessagesModule`
- `ExportsModule`
- `RecommendationsModule`
- `PrismaModule`

Η λογική κάθε περιοχής κρατήθηκε στο αντίστοιχο service. Για παράδειγμα, οι κανόνες των εκδηλώσεων βρίσκονται στο `EventsService` , ενώ τα endpoints βρίσκονται στο `EventsController` . Με αυτόν τον τρόπο ο controller μένει πιο απλός και το service περιέχει την επιχειρησιακή λογική.

Στο frontend χρησιμοποιήθηκαν Angular standalone components. Υπάρχουν ξεχωριστές σελίδες για welcome, login, register, events, event details, create/manage event, bookings, messages, profile και admin users.

3. Σχεδίαση Βάσης Δεδομένων

Το σχήμα της βάσης βρίσκεται στο `backend/prisma/schema.prisma`. Η βάση σχεδιάστηκε γύρω από τις βασικές οντότητες της εκφώνησης.

Τα κύρια μοντέλα είναι:

- `User`: στοιχεία χρήστη, στοιχεία εγγραφής, ρόλοι και κατάσταση έγκρισης.
- `Event`: στοιχεία εκδήλωσης, organizer, status και χωρητικότητα.
- `TicketType`: τύποι εισιτηρίων κάθε εκδήλωσης.
- `Category`: διαθέσιμες κατηγορίες.
- `EventCategory`: ενδιάμεσος πίνακας για τη σχέση many-to-many μεταξύ events και categories.
- `Booking`: κρατήσεις χρηστών.
- `Photo`: φωτογραφίες εκδηλώσεων ως paths ή URLs.
- `Message`: μηνύματα μεταξύ χρηστών.
- `EventView`: προβολές εκδηλώσεων για το σύστημα συστάσεων.

Βασικά enums:

- `RegistrationStatus`: `PENDING`, `APPROVED`, `REJECTED`
- `EventStatus`: `DRAFT`, `PUBLISHED`, `COMPLETED`, `CANCELLED`
- `BookingStatus`: `PENDING`, `CONFIRMED`, `CANCELLED`

Σχεδιαστικές επιλογές

Δεν διαγράφουμε φυσικά χρήστες από τη βάση. Αντί για αυτό χρησιμοποιούμε soft delete με το πεδίο `active`, ώστε να μη χαθούν σχέσεις με κρατήσεις, εκδηλώσεις ή μηνύματα.

Οι κατηγορίες μπήκαν σε ξεχωριστό πίνακα επειδή μία εκδήλωση μπορεί να έχει περισσότερες από μία κατηγορίες. Αυτό υλοποιείται με τον πίνακα `EventCategory`.

Οι φωτογραφίες δεν αποθηκεύονται ως binary δεδομένα στη βάση. Η βάση κρατά μόνο το path ή URL της φωτογραφίας. Αυτό κρατά τη βάση πιο απλή και ελαφριά.

Για τις κρατήσεις χρησιμοποιούνται δύο μεγέθη: `TicketType.available` και `Event.reservedSeats`. Έτσι μπορούμε να ξέρουμε και πόσα εισιτήρια έχουν μείνει ανά τύπο και πόσες συνολικές θέσεις έχουν δεσμευτεί στην εκδήλωση.

4. Πιστοποίηση και Εξουσιοδότηση

Η πιστοποίηση γίνεται με username και password. Οι κωδικοί αποθηκεύονται ως hash με `bcrypt`, όχι σε απλή μορφή.

Μετά από επιτυχημένο login, το backend εκδίδει JWT token. Το token περιέχει:

- το id του χρήστη στο πεδίο `sub`,
- το username,
- τους ρόλους του χρήστη.

Το frontend αποθηκεύει το token στο `localStorage` και το στέλνει αυτόματα στα protected requests μέσω HTTP interceptor.

Για προστασία endpoints χρησιμοποιούνται:

- `JwtAuthGuard`, για να επιτρέπεται πρόσβαση μόνο σε συνδεδεμένους χρήστες.
- `RolesGuard`, για endpoints που απαιτούν συγκεκριμένο ρόλο, όπως admin-only λειτουργίες.

Το frontend ελέγχει επίσης αν το token έχει λήξει, ώστε να εμφανίζει σωστά το navigation bar και να μη μένει ο χρήστης σε σελίδες που απαιτούν σύνδεση ενώ το token δεν ισχύει πλέον. Αυτό είναι κυρίως για καλύτερο user experience, αφού η πραγματική ασφάλεια γίνεται στο backend.

Για local development χρησιμοποιήθηκε HTTPS με self-signed certificates στον φάκελο `certs/`. Σε production περιβάλλον θα χρειαζόταν κανονικό SSL/TLS certificate από hosting provider ή reverse proxy.

5. Διαχείριση Χρηστών

Η εγγραφή νέου χρήστη απαιτεί username, password, επιβεβαίωση password, όνομα, επώνυμο, email, τηλέφωνο, διεύθυνση, πόλη, χώρα, γεωγραφική τοποθεσία και ΑΦΜ.

Όταν ένας χρήστης κάνει register, η αίτησή του μπαίνει αρχικά σε κατάσταση `PENDING`. Αυτό σημαίνει ότι δεν αποκτά πλήρη πρόσβαση μέχρι να εγκριθεί από τον admin.

Ο admin μπορεί:

- να δει λίστα χρηστών,
- να ανοίξει λεπτομέρειες κάθε χρήστη,
- να εγκρίνει ή να απορρίψει νέες εγγραφές,
- να ενημερώσει στοιχεία χρήστη,

- να απενεργοποιήσει χρήστη με soft delete.

Όταν ο admin κάνει login, αν δεν υπάρχει `returnUrl`, οδηγείται στη σελίδα `/admin/users`. Οι απλοί χρήστες οδηγούνται στη σελίδα `/events`.

Στα public user responses δεν επιστρέφεται ποτέ `passwordHash`. Για το login υπάρχει ξεχωριστή εσωτερική αναζήτηση χρήστη, ώστε το auth service να μπορεί να ελέγξει τον κωδικό χωρίς να εκτίθεται το hash στο frontend.

6. Διαχείριση Εκδηλώσεων

Οι εγκεκριμένοι χρήστες μπορούν να δημιουργούν εκδηλώσεις. Κάθε νέα εκδήλωση ξεκινά σε κατάσταση `DRAFT`. Μετά ο διοργανωτής μπορεί να τη συμπληρώσει, να προσθέσει ticket types, categories και φωτογραφίες και τελικά να τη δημοσιεύσει.

Για κάθε εκδήλωση κρατάμε:

- τίτλο
- κατηγορίες
- τύπο εκδήλωσης
- venue, address, city, country
- latitude και longitude,
- ημερομηνία/ώρα έναρξης και λήξης
- capacity
- ticket types
- reserved seats
- status
- description
- photos

Το `organizerId` δεν έρχεται από το request body. Παίρνεται από το JWT του συνδεδεμένου χρήστη. Αυτό έγινε για να μην μπορεί κάποιος να δημιουργήσει event εκ μέρους άλλου χρήστη.

Κανόνες εκδηλώσεων

Η εφαρμογή ελέγχει ότι:

- η ημερομηνία λήξης είναι μετά την ημερομηνία έναρξης
- η χωρητικότητα είναι θετικός αριθμός
- η χωρητικότητα δεν γίνεται μικρότερη από τις ήδη δεσμευμένες θέσεις
- μόνο organizer ή admin μπορεί να διαχειριστεί μία εκδήλωση
- cancelled και completed events δεν αλλάζουν

- η δημοσίευση απαιτεί τουλάχιστον μία κατηγορία
- η δημοσίευση απαιτεί τουλάχιστον ένα ticket type
- το συνολικό quantity των ticket types πρέπει να είναι ίσο με το capacity πριν το publish
- δεν μπορεί να δημιουργηθεί ή να ενημερωθεί ticket type αν το συνολικό quantity ξεπερνά το capacity
- δεν μπορεί να μειωθεί ticket quantity κάτω από τα ήδη κρατημένα εισιτήρια
- delete επιτρέπεται μόνο πριν γίνει η πρώτη κράτηση
- published events με κρατήσεις ακυρώνονται αντί να διαγράφονται

Το `CANCELLED` θεωρήθηκε τελική ιστορική κατάσταση. Δηλαδή ένα ακυρωμένο event δεν επιστρέφει σε `PUBLISHED`. Αυτό κρατά πιο καθαρό το ιστορικό και αποφεύγει μπερδέματα με κρατήσεις που είχαν ήδη ακυρωθεί.

7. Κρατήσεις Εισιτηρίων

Οι επισκέπτες μπορούν να δουν events, αλλά για κράτηση πρέπει να συνδεθούν. Αν κάποιος πατήσει “Sign in to book”, γίνεται redirect στο login και μετά επιστρέφει στο booking page του ίδιου event.

Η διαδικασία κράτησης έχει τρία βήματα:

1. επιλογή ticket type και πλήθους εισιτηρίων
2. σελίδα επιβεβαίωσης
3. τελική ολοκλήρωση κράτησης

Πριν την οριστική υποβολή εμφανίζεται σύνοψη με ticket type, πλήθος εισιτηρίων και συνολικό κόστος. Μετά την επιβεβαίωση η κράτηση θεωρείται οριστική.

Στο backend η κράτηση γίνεται μέσα σε transaction. Αυτό ήταν σημαντικό γιατί πρέπει ταυτόχρονα:

- να μειωθεί το `available` του ticket type
- να αυξηθεί το `reservedSeats` του event
- να δημιουργηθεί η εγγραφή booking

Αν κάτι αποτύχει, γίνεται rollback και δεν μένει η βάση σε μισοενημερωμένη κατάσταση.

Η εφαρμογή ελέγχει ότι:

- το event είναι `PUBLISHED`
- το event δεν έχει ξεκινήσει
- ο χρήστης είναι εγκεκριμένος
- ο χρήστης δεν κάνει booking στο δικό του event

- υπάρχουν αρκετά εισιτήρια για τον συγκεκριμένο ticket type
- δεν ξεπερνιέται η συνολική χωρητικότητα

Το συνολικό κόστος υπολογίζεται στο backend από την τιμή της βάσης και όχι από τιμή που θα μπορούσε να στείλει ο client.

8. Σύστημα Μηνυμάτων

Μετά από κράτηση, ο συμμετέχων και ο διοργανωτής μπορούν να επικοινωνούν μέσω του συστήματος μηνυμάτων.

Υλοποιήθηκαν:

- inbox
- sent messages
- προβολή λεπτομερειών μηνύματος
- σύνταξη νέου μηνύματος
- ένδειξη unread μηνυμάτων στο navbar
- soft delete μηνύματος για κάθε χρήστη ξεχωριστά

Η αποστολή μηνύματος δεν είναι ελεύθερη προς οποιονδήποτε χρήστη. Επιτρέπεται μόνο όταν οι δύο χρήστες συνδέονται μέσω booking. Αυτό σημαίνει ότι ένας συμμετέχων μπορεί να στείλει μήνυμα στον organizer ενός event που έχει κλείσει και ο organizer μπορεί να επικοινωνήσει με συμμετέχοντες του event.

Σε περίπτωση ακύρωσης event, το backend στέλνει ενημερωτικό μήνυμα στους χρήστες που είχαν κάνει κράτηση. Οι κρατήσεις μπορούν να σημειωθούν ως `CANCELLED`, αλλά παραμένουν στη βάση για λόγους ιστορικότητας.

9. Αναζήτηση, Χάρτες και Φωτογραφίες

Η σελίδα `/events` χρησιμοποιείται για browsing και αναζήτηση δημοσιευμένων εκδηλώσεων. Υποστηρίζει φίλτρα για:

- keyword
- πόλη
- τύπο εκδήλωσης
- κατηγορία
- ημερομηνία από/έως
- ελάχιστη και μέγιστη τιμή εισιτηρίου

Το keyword ψάχνει σε τίτλο, περιγραφή, venue, address, city και country. Αυτό το κάναμε ώστε η αναζήτηση να μοιάζει πιο φυσική στον χρήστη και να μη χρειάζεται να ξέρει σε ποιο ακριβώς πεδίο υπάρχει η πληροφορία.

Τα αποτελέσματα είναι σελιδοποιημένα. Το frontend ζητά συγκεκριμένη σελίδα με `page` και `limit`, ενώ το backend επιστρέφει τα `items`, το συνολικό πλήθος και το `totalPages`.

Για τους χάρτες χρησιμοποιείται OpenStreetMap. Στο create/manage event η εφαρμογή μπορεί να βρει συντεταγμένες από `address + city + country`. Δεν χρησιμοποιούμε το venue στο geocoding, γιατί τα ονόματα χώρων μπορεί να μπερδεύουν περισσότερο την αναζήτηση. Στη σελίδα λεπτομερειών εμφανίζεται embedded χάρτης όταν υπάρχουν latitude και longitude.

Για τις φωτογραφίες υποστηρίζονται URL/path references και upload εικόνας. Το backend αποθηκεύει το αρχείο στον φάκελο `uploads` και στη βάση κρατά το path. Στο browsing εμφανίζεται η πρώτη φωτογραφία του event. Αν δεν υπάρχει φωτογραφία, εμφανίζεται placeholder με τον τύπο του event.

10. Εξαγωγή Δεδομένων σε XML/JSON

Ο admin μπορεί να εξάγει τις εκδηλώσεις σε XML και JSON. Τα κουμπιά εμφανίζονται στο events page μόνο για admin χρήστες. Παρ' όλα αυτά, ο βασικός έλεγχος γίνεται και στο backend με `JwtAuthGuard` και `RolesGuard`, ώστε να μην αρκεί απλώς να κρυφτεί το κουμπί από το UI.

Το XML ακολουθεί τη δομή του DTD της εκφώνησης. Περιλαμβάνει:

- event στοιχεία
- categories
- geolocation
- ticket types
- bookings
- organizer
- status
- description
- media/photos

Το JSON περιέχει αντίστοιχη δομή, πιο κατάλληλη για χρήση από εφαρμογές ή debugging.

11. Σύστημα Συστάσεων

Το σύστημα συστάσεων υλοποιήθηκε στο backend με Biased Matrix Factorization, χωρίς χρήση έτοιμης βιβλιοθήκης recommendation engine. Στόχος του είναι κάθε συνδεδεμένος χρήστης να βλέπει προτεινόμενες εκδηλώσεις στην αρχική σελίδα και στο `/events`, μέσα από την επιλογή "Recommended".

Η βασική ιδέα είναι ότι κάθε χρήστης και κάθε εκδήλωση αναπαρίστανται με ένα μικρό διάνυσμα αριθμών. Το μοντέλο μαθαίνει αυτά τα διανύσματα από τη συμπεριφορά των χρηστών και μετά προβλέπει πόσο πιθανό είναι να ενδιαφέρει μία εκδήλωση έναν συγκεκριμένο χρήστη.

Ο τύπος που χρησιμοποιείται είναι:

```
r_hat = μ + b_u + b_i + p_u · q_i
```

Όπου:

- `μ` είναι ο γενικός μέσος όρος των βαθμολογιών
- `b_u` είναι το bias του χρήστη
- `b_i` είναι το bias της εκδήλωσης
- `p_u` είναι το διάνυσμα του χρήστη
- `q_i` είναι το διάνυσμα της εκδήλωσης
- `p_u · q_i` είναι το εσωτερικό γινόμενο των δύο διανυσμάτων

Τα biases βοηθούν το μοντέλο να ξεχωρίζει τη γενική δημοφιλία από την πραγματική εξατομίκευση. Για παράδειγμα, μία εκδήλωση μπορεί να είναι γενικά δημοφιλής, αλλά το διάνυσμα του χρήστη δείχνει αν ταιριάζει στα δικά του ενδιαφέροντα.

Δεδομένα και σήματα ενδιαφέροντος

Η εφαρμογή δεν έχει βαθμολογίες με αστέρια, οπότε χρησιμοποιήσαμε έμμεσα σήματα ενδιαφέροντος:

- κράτηση μη ακυρωμένης εκδήλωσης: βαθμός `5`
- δύο ή περισσότερες προβολές της ίδιας εκδήλωσης: βαθμός `4`
- μία προβολή εκδήλωσης: βαθμός `3`
- αρνητικό δείγμα από εκδήλωση που ο χρήστης δεν έχει αγγίξει: βαθμός `1`

Για κάθε ζεύγος χρήστη-εκδήλωσης κρατείται μόνο μία βαθμολογία, η μεγαλύτερη. Για παράδειγμα, αν κάποιος είδε μία εκδήλωση και μετά έκανε κράτηση, το τελικό σήμα είναι κράτηση με βαθμό `5`.

Οι προβολές εκδηλώσεων αποθηκεύονται στον πίνακα `EventView`. Καταγράφονται μόνο όταν ο χρήστης είναι συνδεδεμένος, όταν η εκδήλωση είναι `PUBLISHED` και όταν

δεν είναι ο ίδιος ο διοργανωτής της εκδήλωσης. Το frontend καλεί το endpoint `POST /events/:id/view` όταν φορτώνει η σελίδα λεπτομερειών ενός event.

Χρήση dataset

Για να υπάρχει αρκετό υλικό εκπαίδευσης, χρησιμοποιήθηκε και το dataset του e-class, τοπικά στον φάκελο των δικών μας μηχανημάτων.

```
backend/private-data/recommendations/dataset/rel_event_csvs/
```

Δεν μπαίνει στο git, επειδή είναι μεγάλο και δεν είναι μέρος του πηγαίου κώδικα. Για τον ίδιο λόγο αγνοείται και το εκπαιδευμένο μοντέλο.

Η αρχική ιδέα ήταν να χρησιμοποιηθούν τα IDs του dataset ξεχωριστά, αλλά αυτό δεν βοηθούσε αρκετά τις πραγματικές εκδηλώσεις της εφαρμογής. Για αυτό τελικά δημιουργήθηκε seed script που εισάγει ένα υποσύνολο του dataset στη δική μας βάση ως κανονικούς χρήστες, events, bookings και event views.

Το script κρατά τις πιο δημοφιλείς εκδηλώσεις του dataset και χρήστες που έχουν τουλάχιστον δύο αλληλεπιδράσεις. Οι παλιές ημερομηνίες του dataset μετατοπίζονται ώστε τα events να είναι μελλοντικά και να μπορούν να χρησιμοποιηθούν από την εφαρμογή. Για κάθε dataset event δημιουργείται ένας γενικός ticket type.

Η εντολή είναι:

```
cd backend
npm run prisma:recommender-seed
```

Στο συγκεκριμένο seed δημιουργούνται dataset users με username τύπου `ds_<id>`, ένας κοινός organizer `ds_organizer`, δημοσιευμένες εκδηλώσεις, κρατήσεις και προβολές. Έτσι το recommendation model εκπαιδεύεται πάνω σε πραγματικές οντότητες της ίδιας βάσης και όχι σε ξεχωριστά, άσχετα IDs.

Εκπαίδευση μοντέλου

Η εκπαίδευση γίνεται με Stochastic Gradient Descent (SGD). Οι βασικές παράμετροι της υλοποίησης είναι:

- 16 latent factors
- 60 epochs
- learning rate `0.02`

- L2 regularization `0.05`
- 3 αρνητικά δείγματα για κάθε θετικό
- 50 epochs για fold-in χρήστη

Χρησιμοποιείται negative sampling επειδή τα περισσότερα σήματα στην εφαρμογή είναι θετικά. Αν το μοντέλο έβλεπε μόνο κρατήσεις και προβολές, θα μάθαινε να προβλέπει υψηλό ενδιαφέρον σχεδόν για όλα. Με τα αρνητικά δείγματα μαθαίνει και ποιες εκδηλώσεις είναι λιγότερο πιθανό να ενδιαφέρουν τον χρήστη.

Το μοντέλο εκπαιδεύεται στην εκκίνηση του server και επανεκπαιδεύεται περιοδικά κάθε 10 λεπτά. Επίσης μπορεί να εκπαιδευτεί χειροκίνητα με:

```
cd backend
npm run recommendations:train
```

Η εντολή αποθηκεύει το μοντέλο στο:

```
backend/private-data/recommendations/model.json
```

Το αρχείο αυτό περιέχει τους πίνακες/χάρτες του μοντέλου σε μορφή JSON. Στην εκκίνηση, αν υπάρχει ήδη `model.json`, φορτώνεται ώστε η εφαρμογή να έχει διαθέσιμο μοντέλο ακόμα και πριν ολοκληρωθεί νέο training.

Fold-in και νέοι χρήστες

Ένα σημαντικό σημείο είναι ότι οι προτάσεις πρέπει να δουλεύουν και για χρήστες που δεν υπήρχαν στην τελευταία εκπαίδευση ή δεν έχουν κρατήσεις. Για αυτό χρησιμοποιείται fold-in.

Σε κάθε αίτημα `GET /recommendations`, το backend παίρνει τις τωρινές αλληλεπιδράσεις του χρήστη και υπολογίζει εκείνη τη στιγμή το δικό του bias και διάνυσμα, κρατώντας σταθερά τα διανύσματα των εκδηλώσεων. Έτσι:

- ένας χρήστης με κρατήσεις παίρνει προτάσεις από κρατήσεις και προβολές
- ένας χρήστης χωρίς κρατήσεις παίρνει προτάσεις μόνο από τις εκδηλώσεις που έχει δει
- ένας χρήστης χωρίς καμία αλληλεπίδραση παίρνει fallback προτάσεις με βάση τη δημοφιλία του μοντέλου

Αυτό καλύπτει και την απαίτηση της εκφώνησης ότι, αν ο χρήστης δεν έχει ιστορικό κρατήσεων, το σύστημα πρέπει να λειτουργεί με βάση τις εκδηλώσεις που έχει επισκεφθεί.

Παραγωγή προτάσεων

Το endpoint `GET /recommendations` είναι protected με JWT. Για κάθε χρήστη το backend βρίσκει υποψήφιες εκδηλώσεις που είναι:

- `PUBLISHED`
- μελλοντικές
- όχι δικές του
- όχι ήδη κλεισμένες ή ιδωμένες από αυτόν
- όχι γεμάτες
- με τουλάχιστον ένα ticket type διαθέσιμο

Μετά υπολογίζεται score για κάθε υποψήφια εκδήλωση και επιστρέφονται οι 6 καλύτερες. Εκδηλώσεις που δεν υπάρχουν ακόμα στο μοντέλο, επειδή δεν έχουν καμία αλληλεπίδραση, μπαίνουν στο τέλος της κατάταξης. Αν δεν υπάρχει μοντέλο, χρησιμοποιείται fallback με βάση τις περισσότερες κρατημένες θέσεις και την πιο κοντινή ημερομηνία.

Αξιολόγηση

Για την αξιολόγηση χρησιμοποιείται leave-one-out. Για κάθε χρήστη με τουλάχιστον δύο αλληλεπιδράσεις, κρύβεται μία και ελέγχεται αν το μοντέλο την επαναφέρει μέσα στις top-10 προτάσεις. Το metric είναι `HR@10`.

Η χειροκίνητη εκπαίδευση τυπώνει επίσης RMSE ανά epoch, ώστε να φαίνεται ότι το SGD συγκλίνει. Ενδεικτικά αποτελέσματα που μετρήθηκαν στη βάση μας ήταν:

```
HR@10 model:      περίπου 51%
HR@10 popularity: περίπου 22%
HR@10 random:     περίπου 3.5%
```

Το αποτέλεσμα αλλάζει λίγο από εκτέλεση σε εκτέλεση, επειδή υπάρχουν τυχαία αρνητικά δείγματα. Παρ' όλα αυτά, το μοντέλο ήταν αισθητά καλύτερο από τη σκέτη δημοφιλία και πολύ καλύτερο από τυχαίες προτάσεις.

Υπάρχουν και tests για το recommendation service. Τα tests ελέγχουν ότι οι βαθμολογίες δημιουργούνται σωστά, ότι το RMSE μειώνεται, ότι οι προτάσεις μένουν στην ομάδα ενδιαφέροντος του χρήστη, ότι το cold start με μόνο views δουλεύει, ότι το fallback δουλεύει χωρίς μοντέλο και ότι το query εξαιρεί σωστά events που δεν πρέπει να προταθούν.

12. Εγκατάσταση και Εκτέλεση

Προϋποθέσεις

Χρειάζονται:

- Node.js / npm συμβατά με τα `engines` του project
- PostgreSQL βάση
- συμπληρωμένο `.env` στο backend
- certificates στον φάκελο `certs/` για local HTTPS

Backend

Από το root του project:

```
cd backend
npm install
```

Ενδεικτικό `.env`:

```
DATABASE_URL="postgresql://..."
DIRECT_DATABASE_URL="postgresql://..."
JWT_SECRET="..."
```

Για Prisma και βασικό seed:

```
npx prisma generate
npx prisma migrate dev
npm run prisma:seed
```

Για να φορτωθεί το dataset του recommender στη βάση, αφού έχει τοποθετηθεί στο `private-data`:

```
npm run prisma:recommender-seed
```

Για χειροκίνητη εκπαίδευση και αξιολόγηση του μοντέλου συστάσεων:

```
npm run recommendations:train
```

Για εκκίνηση backend:

```
npm run start:dev
```

Το backend τρέχει σε HTTPS, στη διεύθυνση:

```
https://localhost:3000
```

Frontend

Σε δεύτερο terminal:

```
cd frontend
npm install
npm run start
```

Το frontend τρέχει στο Angular dev server:

```
https://localhost:4200
```

Σημείωση για certificates

Επειδή χρησιμοποιούνται self-signed certificates στο local περιβάλλον, ο browser μπορεί να εμφανίσει προειδοποίηση την πρώτη φορά. Για ανέβασμα σε production περιβάλλον θα πρέπει να χρησιμοποιηθεί κανονικό certificate.

13. Έλεγχος και Δοκιμές

Κατά την ανάπτυξη κάναμε ελέγχους τόσο με browser όσο και με curl/build commands.

Ενδεικτικά ελέγχθηκαν:

- register και login
- admin approval/rejection
- ανακατεύθυνση μετά το login για admin και απλό χρήστη
- προστατευμένα endpoints με και χωρίς JWT
- δημιουργία, ενημέρωση, δημοσίευση, ακύρωση και διαγραφή event
- κανόνες publish για categories και ticket quantities
- διαθεσιμότητα ticket types και χωρητικότητα

- booking flow με confirmation
- my bookings
- organizer event bookings
- messaging inbox/sent/details/compose
- unread messages indicator
- cancellation announcement
- search filters και pagination
- OpenStreetMap geocoding και map preview
- ανέβασμα φωτογραφιών και εμφάνιση φωτογραφιών στο browsing
- XML/JSON exports
- εκπαίδευση του συστήματος συστάσεων και endpoint προτάσεων
- recommender tests με `npx jest recommendations`

Για manual backend testing χρησιμοποιήθηκαν και curl requests, ειδικά πριν συνδεθεί κάθε λειτουργία με το frontend.

14. Συμπεράσματα και Μελλοντικές Βελτιώσεις

Συνολικά υλοποιήθηκε μία λειτουργική εφαρμογή για διαχείριση εκδηλώσεων και κρατήσεων, με ρόλους χρηστών, authentication, admin approval, events, bookings, messages, exports και recommendations.

Οι βασικές δυσκολίες ήταν η σωστή διαχείριση των κανόνων χωρητικότητας, η σύνδεση του frontend με protected backend endpoints, η διαχείριση διαφορετικών ρόλων και η υλοποίηση και αξιολόγηση του recommendation system. Επίσης, χρειάστηκε προσοχή σε λεπτομέρειες όπως η λήξη του JWT, η επιστροφή του χρήστη μετά από login στο booking page, η ακύρωση events με ήδη υπάρχουσες κρατήσεις και η αποφυγή απώλειας ιστορικών δεδομένων.

Οι δυσκολίες αντιμετωπίστηκαν με σταδιακή ανάπτυξη, έλεγχο κάθε endpoint ξεχωριστά και μετά σύνδεση με το frontend. Κρατούσαμε επίσης σημειώσεις στην πορεία, ώστε να υπάρχει καθαρή εικόνα του τι έχει υλοποιηθεί και γιατί.

Πιθανές μελλοντικές βελτιώσεις είναι:

- περισσότερα αυτόματα tests
- ανέβασμα σε production περιβάλλον με κανονικό HTTPS certificate
- καλύτερη εκπαίδευση του μοντέλου συστάσεων με περισσότερα πραγματικά interactions
- βελτίωση UI σε μικρότερες οθόνες

- πιο αναλυτικά στατιστικά για organizer/admin